2.2 Declaring Classes and Creating Objects

Chapter 18 moves from the OOP vocabulary to PHP syntax. We will declare one class per file, load its declaration, create objects, and interact with them through methods.

Declare a class

Create `src/Tutorial/Class2/Course.php`:

```php
<?php

declare(strict_types=1);

class Course
{
    private Person $instructor;
    private string $title;
    private array $registrations = [];

    public function setInstructor(Person $instructor): void
    {
        $this->instructor = $instructor;
    }

    public function getInstructor(): Person
    {
        return $this->instructor;
    }

    public function setTitle(string $title): void
    {
        $this->title = $title;
    }

    public function getTitle(): string
    {
        return $this->title;
    }

    public function formatForDisplay(): string
    {
        return $this->title . ' by ' . $this->instructor->getDisplayName() . ' ('
. count($this->registrations) . ' registrations)';
    }
}
```

`$this` means “the current object”—the particular instance that received the method call.

The class name uses StudlyCaps; methods and properties use camelCase. Modern PHP should declare property, parameter, and return types whenever the intended type is known.

Visibility

Modifier	Accessible from
`public`	any code with access to the object
`protected`	the declaring class and its subclasses
`private`	the declaring class only

Prefer private properties until there is a reason to expose more. Public methods form the class's interface with the rest of the application.

Load the declaration and create an object

Create `public/tutorials/2-2-defining-custom-classes.php`:

```php
<?php

declare(strict_types=1);

require_once __DIR__ . '/../../src/Tutorials/Class2/Course.php';

require_once __DIR__ . '/../../src/Tutorials/Class2/Person.php';

$cis334 = new Course();
$cis334->setTitle('CIS 334: PHP and MySQL');

$instructor = new Person();
$instructor->setDisplayName('Nate LaClaire');
$cis334->setInstructor($instructor);

echo $cis334->formatForDisplay();
```

`require_once` evaluates the class declaration before we use it. `__DIR__` makes the path relative to the file containing the statement. The `new` expression creates an object and returns its identity; `$hammer` stores that object reference.

Each object has its own state

```php
$cis333 = new Course();
$cis333->setTitle('CIS 333: Web Applications: PHP');
$cis333->setInstructor($instructor);

echo $cis333->formatForDisplay();
```

The method code is shared, but each object keeps its own property values. Inside `setTitle()`, `$this` refers to `$cis334` for the first call and `$cis333` for the second.

Add registration behavior to `Course` the same way:

```
    public function addRegistration(Registration $registration): void
    {
        $this->registrations[] = $registration;
    }

    // Instantiate a Registration object for the given student and add it to the
Course
    public function registerStudent(Person $student): void
    {
        $registration = new Registration();
        $registration->student = $student;
        $registration->course = $this;
        $registration->registrationTime = date('Y-m-d H:i:s');
        $this->addRegistration($registration);
    }

    public function getRegistrations(): array
    {
        return $this->registrations;
    }
```

Because `$registrations` is private, code outside the class cannot directly manipulate it. Instead, `addRegistration()` and `registerStudent()` provide controlled ways to add registrations, while `getRegistrations()` allows read-only access to the list of registrations.